HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND ENGINEERING
FACULTY OF ADVANCED EDUCATION

———————————

HCM-UTE

COURSE REPORT

Subject: Embedded Systems

First semester, academic year 2026–2027

PROJECT TOPIC 2: EMBEDDED LINUX SYSTEM DEVELOPMENT

LINUX DEVICE DRIVER FOR AN LCD2004 DISPLAY AND FOUR PUSH BUTTONS ON RASPBERRY PI 5

Supervisor: Assoc. Prof. Dr. Trương Ngọc Sơn
Class code: 261EMSY435664E_01FIE

Members:		
	Nguyễn Phước An	24119007
	Đặng Đình Mạnh	24119055
	Nguyễn Hoàng Nam	24119062
	Võ Minh Tâm	24119082

Ho Chi Minh City, October 2026

LIST OF CONTRIBUTING MEMBERS

SEMESTER I, ACADEMIC YEAR: 2026–2027

GROUP: .....

Topic: Linux device driver for an LCD2004 display and four push buttons on Raspberry Pi 5 (Project Topic 2: Embedded Linux System Development).

Full name	Student ID	Assigned tasks	Completion (%)
Nguyễn Phước An	24119007	Device Tree overlay, Makefiles, loading the driver, building the driver into the kernel	100
Đặng Đình Mạnh	24119055	LCD control over I2C (PCF8574, HD44780), LCD wiring	100
Nguyễn Hoàng Nam	24119062	Button handling: interrupts, debouncing, event queue; button wiring	100
Võ Minh Tâm	24119082	Device file /dev/lcd2004 (write, ioctl, read), test programs and demo	100

Lecturer's comments:

.....

.....

.....

.....

.....

Ho Chi Minh City, .../...../2026

Lecturer

TABLE OF CONTENTS

LIST OF CONTRIBUTING MEMBERS.....i
LIST OF FIGURES.....v
LIST OF TABLES.....vi
LIST OF ABBREVIATIONS.....vii
INTRODUCTION.....1
1. Project overview.....1
2. System objectives.....1
3. Scope.....1
4. Related work.....2
4.1. Course laboratory guide.....2
4.2. Drivers in the Linux kernel.....2
4.3. User-space libraries.....2
4.4. Comparison.....2
5. Practical significance.....2
CHAPTER 1: THEORETICAL BACKGROUND AND HARDWARE.....4
1.1. Mapping to the assignment requirements.....4
1.2. Theoretical background.....4
1.2.1. Kernel module and built-in driver.....4
1.2.2. Device files and misc devices.....4
1.2.3. I2C bus and PCF8574.....4
1.2.4. HD44780 in 4-bit mode.....5
1.2.5. GPIO, interrupts and debouncing.....6
1.2.6. Delayed work, kfifo, wait queue and poll.....6
1.2.7. Device Tree and overlays.....6
1.3. Hardware and components.....6
1.3.1. Bill of materials.....6
1.3.2. Raspberry Pi 5 and RP1.....7
1.3.3. Pin assignment.....7

1.3.4. Wiring diagram.....7
1.3.5. Voltage protection.....8
CHAPTER 2: SYSTEM DESIGN.....9
2.1. System block diagram.....9
2.2. Driver structure.....9
2.3. Device Tree overlay.....10
2.4. Interface for programs.....10
2.5. Button event path.....11
2.6. Sharing and locking.....12
2.7. Start-up (probe) and removal.....12
2.8. Test and demo programs.....13
2.9. Built-in kernel design.....14
CHAPTER 3: IMPLEMENTATION AND SOURCE CODE.....15
3.1. Development environment.....15
3.2. Hardware assembly.....15
3.3. Building and loading the module.....16
3.4. Building the driver into the kernel.....16
3.5. Source code.....16
3.5.1. Sending data to the LCD.....17
3.5.2. write().....18
3.5.3. ioctl().....18
3.5.4. Interrupt and debouncing.....19
3.5.5. read() and poll().....19
3.5.6. probe() and remove().....20
3.5.7. Demo main loop.....20
CHAPTER 4: TESTING AND EVALUATION.....21
4.1. Test method.....21
4.2. Results with the module.....21
4.3. Results with the built-in kernel.....22
4.4. Comparison with the course lab guide.....23

4.5. Overall assessment.....23
4.6. Known limitations.....23
4.7. Difficulties and solutions.....23
CONCLUSION.....25
REFERENCES.....25

LIST OF FIGURES

Figure 1. Raspberry Pi 40-pin GPIO header (source: Raspberry Pi Ltd documentation [6]).....7
Figure 2. Wiring of the Raspberry Pi 5, level shifter, LCD board and buttons.....8
Figure 3. System block diagram.....9
Figure 4. Path of a button event.....11
Figure 5. Steps of lcd_probe().....13
Figure 6. States of the demo program.....14
Figure 7. Steps to build the driver into the kernel.....14
Figure 8. The assembled hardware.....15
Figure 9. Welcome screen from lcd_probe().....21
Figure 10. lcd_test result.....22
Figure 11. Demo controlled by the buttons.....22
Figure 12. Terminal output of the HW-8 checks.....23

LIST OF TABLES

Table 1. Comparison of related approaches....2
Table 2. Assignment requirements and solutions....4
Table 3. PCF8574 pins and LCD signals....5
Table 4. HD44780 commands used by the driver....5
Table 5. Components....6
Table 6. Raspberry Pi 5 pins used in the project....7
Table 7. Parts of the driver....9
Table 8. Operations of /dev/lcd2004....10
Table 9. Shared data and locks....12
Table 10. User programs....13
Table 11. Module and built-in....14
Table 12. Development tools....15
Table 13. Test steps and results....21
Table 14. Checks of the built-in driver (HW-8)....22
Table 15. Our choices compared with the lab guide....23
Table 16. Difficulties and solutions....23

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
BCM	Broadcom pin numbering of the Raspberry Pi GPIO header
DDRAM	Display Data RAM of the HD44780 controller
DT / DTS / DTBO	Device Tree / Device Tree Source / Device Tree Blob Overlay
GPIO	General-Purpose Input/Output
I2C	Inter-Integrated Circuit, a two-wire serial bus
IRQ	Interrupt Request
ISR	Interrupt Service Routine
kfifo	Kernel first-in first-out ring buffer
LCD	Liquid Crystal Display
OS	Operating System
RP1	Raspberry Pi I/O controller chip of the Raspberry Pi 5
SCL / SDA	Serial Clock / Serial Data lines of I2C
SMBus	System Management Bus, a subset of I2C
SoC	System on Chip
SSH	Secure Shell
WSL	Windows Subsystem for Linux

INTRODUCTION

This report describes a Linux driver for a 20×4 character LCD (LCD2004) and four push buttons on a Raspberry Pi 5. It is Project Topic 2 (Embedded Linux System Development) of the course Embedded Systems.

The driver controls the LCD over I2C and reads the buttons with interrupts. Programs use both devices through one file: `/dev/lcd2004`. The driver works in two forms: as a module (`.ko`) and built into the kernel.

1. Project overview

On a microcontroller, the program drives the LCD and reads the buttons directly. On Linux this is not allowed: normal programs cannot touch the hardware. They must go through a driver in the kernel. The driver handles the hardware details and gives programs simple file operations: `open`, `read`, `write`, `ioctl` and `poll`.

The assignment asks for a driver for an LCD or a 7-segment display together with push buttons, plus test programs and a demo where the buttons control the display. Our group chose the LCD and buttons. The hardware is a Raspberry Pi 5, an LCD2004 with a PCF8574 I2C board, a BSS138 level shifter and a board with four buttons.

2. System objectives

- Show text on all four rows of the LCD.
- Read four buttons with interrupts. One press gives exactly one event.
- Give programs one device file, `/dev/lcd2004`: `write()` prints text, `ioctl()` clears the screen, moves the cursor and turns the backlight on or off, `read()` and `poll()` wait for buttons.
- Describe the I2C address and button pins in a Device Tree overlay, not in the code.
- Build the driver as a module and load it (“meets requirements” level).
- Build the same driver into the kernel (“good” level).
- Write two test programs and one demo program.
- Protect the 3.3 V pins of the Raspberry Pi from the 5 V LCD board.

3. Scope

The project covers the circuit, the driver, the Device Tree overlay, building the module and the kernel, three user programs and the tests on real hardware. The driver shows normal ASCII characters only (no Vietnamese accents), supports one LCD and exactly four buttons. Scrolling text is done by the demo program, not by the driver. Everything was tested on one Raspberry Pi 5 (4 GB) with Raspberry Pi OS 64-bit and kernel 6.18.

4. Related work

4.1. Course laboratory guide

The lab guide of the course (exercises 2.1–2.5) [2] writes drivers for an LED, a button and an LCD1602 on a Raspberry Pi 4. It uses older kernel functions: fixed GPIO numbers, a fixed I2C address in the code, and debouncing inside the interrupt handler. It is easy to follow but does not move well to the Raspberry Pi 5.

4.2. Drivers in the Linux kernel

Linux already has a general HD44780 LCD driver [14] and a general button driver called `gpio-keys` [15]. They work well, but the LCD and the buttons become two separate devices, and the LCD is controlled with special escape codes.

4.3. User-space libraries

Many hobby projects control the LCD from a normal program (for example a Python library writing to `/dev/i2c-1`). No driver is needed, so this does not meet the assignment. Also, two programs cannot share the LCD safely.

4.4. Comparison

Table 1 compares these ways with our project.

Table 1. Comparison of related approaches

Approach	Display	Buttons	How programs use it
Course lab guide [2]	LCD1602, address fixed in code	1 button, debounce in the interrupt handler	`read()` returns a status text
Kernel `hd44780` + `gpio-keys` [14], [15]	General LCD driver	Separate input driver	Escape codes, input events
User-space library	Program writes to `/dev/i2c-1`	Program checks the pins in a loop	Library calls, no driver
This project	LCD2004, address in Device Tree	4 buttons, interrupt + debounce, event queue	One file: `write`, `ioctl`, `read`, `poll`

Our driver uses the modern kernel style but keeps the LCD and the buttons in one small driver, so every member can explain the whole path from a button press to a character on the screen.

5. Practical significance

The project shows the full chain of an embedded Linux product: circuit, Device Tree, driver, kernel build and programs. The same steps work for other small I2C or GPIO

devices such as sensors or relays. The built-in kernel shows how a product can have its device working right after boot. The parts are cheap, and the source code is public [17].

CHAPTER 1: THEORETICAL BACKGROUND AND HARDWARE

1.1. Mapping to the assignment requirements

Table 2 shows each requirement and where it is answered.

Table 2. Assignment requirements and solutions

Requirement	Solution	Section
Driver for an LCD with push buttons	Driver `lcd2004_btn`: LCD2004 over I2C, four buttons with interrupts	2.2–2.7
Driver as a module, installed (meets requirements)	`lcd2004_btn.ko`, loaded with `insmod` (tests HW-2, HW-7)	3.3, 4.2
Driver built into the kernel (good)	Kernel `6.18.57-v8-16k-lcd` with the driver inside (test HW-8)	2.9, 3.4, 4.3
Test programs	`lcd_test` (LCD) and `button_test` (buttons)	2.8, 4.2
Demo: buttons as input, LCD as output	`demo`: greeting, member list, member details	2.8
Run on an embedded board	Raspberry Pi 5, Raspberry Pi OS 64-bit	1.3

1.2. Theoretical background

1.2.1. Kernel module and built-in driver

A driver can be used in two ways:

- **Module (`.ko`):** a separate file. It is loaded with `insmod` and removed with `rmmod`. It must be built for the exact kernel version that is running.
- **Built-in:** the driver is part of the kernel file. It starts during boot and cannot be removed.

The choice is made in the kernel settings (Kconfig): `=m` gives a module, `=y` gives a built-in driver. Our code uses the macro `module_i2c_driver()`, which works in both cases, so the code did not change at all.

1.2.2. Device files and misc devices

Programs talk to a driver through a file in `/dev`. The driver gives the kernel a list of functions (`read`, `write`, `ioctl`, `poll`) to call when a program uses that file. The usual way to create the file takes four steps. A *misc device* does it in one call, `misc_register()`. Our driver uses it to create `/dev/lcd2004`.

1.2.3. I2C bus and PCF8574

I2C uses two wires: SDA (data) and SCL (clock). Each device on the bus has an address. The Raspberry Pi is the master and starts every transfer [5].

The PCF8574 is a small chip that turns I2C into 8 output pins [4]. When the Pi writes one byte, the 8 bits appear on the 8 pins P0–P7. On the LCD board these pins are wired to the LCD as in Table 3. Our board uses address 0x27. Some boards use 0x3F, so the address can be changed in the overlay.

Table 3. PCF8574 pins and LCD signals

PCF8574 pin	LCD signal	Meaning
P0	RS	0 = command, 1 = character
P1	RW	Always 0 (we only write)
P2	E	Enable: a pulse tells the LCD to read the data
P3	Backlight	1 = backlight on
P4–P7	D4–D7	4 data bits

1.2.4. HD44780 in 4-bit mode

The HD44780 is the controller chip inside the LCD [3]. The PCF8574 has only 8 pins, so only 4 data lines are used. Each byte is sent in two halves (high 4 bits, then low 4 bits). Each half is taken by the LCD when E goes from 1 to 0.

At power-on, the datasheet asks for a fixed start-up order: wait 40 ms, send 0x3 three times, then 0x2 to enter 4-bit mode. After that, normal commands can be sent (Table 4).

Table 4. HD44780 commands used by the driver

Code	Command	Meaning
0x28	Function set	4-bit mode, 2 lines, 5×8 font
0x08	Display off	Turn the display off during start-up
0x01	Clear display	Clear the screen (takes about 1.5 ms)
0x06	Entry mode	Move the cursor right after each character
0x0C	Display on	Display on, no cursor
0x80 + address	Set address	Move the cursor to a row and column

The rows of an LCD2004 are not in order in memory. Row 0 starts at address 0x00, row 1 at 0x40, row 2 at 0x14 and row 3 at 0x54. So after row 0 the LCD jumps to row 2. To avoid this, the driver sets the address itself at the start of each row.

Speed: one I2C write takes about 0.2 ms, and one character needs 4 writes, so about 0.9 ms per character. A full screen (80 characters) takes about 75 ms. This is fast enough for text that moves every 200 ms.

1.2.5. GPIO, interrupts and debouncing

The driver gets the button pins from the Device Tree with `devm_gpiod_get_array()` [10]. The pins are marked *active-low*: a pressed button pulls the pin to 0 V, but the driver reads 1. Each pin has an interrupt on both edges, so the driver knows when a button is pressed and when it is released.

A button contact *bounces*: for a few milliseconds it opens and closes many times. Without care, one press would look like many presses. Our fix: every edge restarts a 30 ms timer. The pin is read only after 30 ms with no new edge. An event is made only when the button changes from released to pressed.

1.2.6. Delayed work, kfifo, wait queue and poll

- **Delayed work:** an interrupt handler must be very short. So it only starts a timer; the real work runs later in a kernel thread.
- **kfifo:** a small queue (16 bytes) that stores button events until a program reads them. A spinlock protects it because two CPU cores can use it at once.
- **Wait queue:** when a program calls `read()` and there is no event, it sleeps. A new event wakes it up. No CPU time is wasted.
- **poll:** lets a program wait for a button *or* a timeout in one call.

1.2.7. Device Tree and overlays

On ARM boards the kernel does not find devices by itself. It reads a list of devices called the Device Tree [9]. Each entry has a `compatible` name (which driver to use), an address and other data such as pins. An *overlay* is a small addition to this list. The Raspberry Pi loads it at boot when `config.txt` has a `dtoverlay=` line. When the kernel finds our entry, it calls the driver's `probe()` function.

1.3. Hardware and components

1.3.1. Bill of materials

Table 5. Components

Component	Qty	Notes
Raspberry Pi 5 (4 GB) with 5 V/5 A power supply	1	Raspberry Pi OS 64-bit
LCD2004 with PCF8574 I2C board	1	5 V supply, address 0x27
4-channel level shifter (BSS138)	1	2 channels used for SDA and SCL
Button board (pins V, 1, 2, 3, 4, G)	1	4 buttons, common ground
Breadboard and jumper wires	1 set	Short I2C wires (under 20 cm)
Multimeter	1	Check wiring before power-on

1.3.2. Raspberry Pi 5 and RP1

On the Raspberry Pi 5, the GPIO and I2C pins are controlled by a separate chip called RP1 [6], [7]. This has two effects:

- Old tutorials that write directly to hardware registers do not work. Our driver only uses kernel functions.
- The pin number "17" in old code is not the real kernel number on the Pi 5. Our driver takes the pins from the Device Tree, so this problem does not happen.

1.3.3. Pin assignment

Table 6. Raspberry Pi 5 pins used in the project

Pin	Name	Connected to
1	3V3	LV of the level shifter
2	5V	VCC of the LCD board and HV of the level shifter
3	GPIO2 (SDA)	LV1 → HV1 → SDA of the LCD board
5	GPIO3 (SCL)	LV2 → HV2 → SCL of the LCD board
6	GND	GND of the LCD and the level shifter
9	GND	G of the button board
11	GPIO17	Button 1
13	GPIO27	Button 2
15	GPIO22	Button 3
16	GPIO23	Button 4
17	3V3	V of the button board

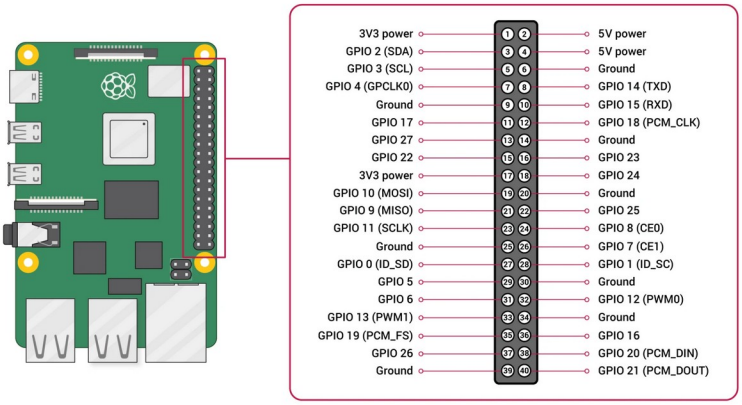


Figure 1. Raspberry Pi 40-pin GPIO header (source: Raspberry Pi Ltd documentation [6])

1.3.4. Wiring diagram

Figure 2 shows the full circuit. Each button connects its pin to ground when pressed. When released, the pin is kept at 3.3 V by an internal pull-up resistor.

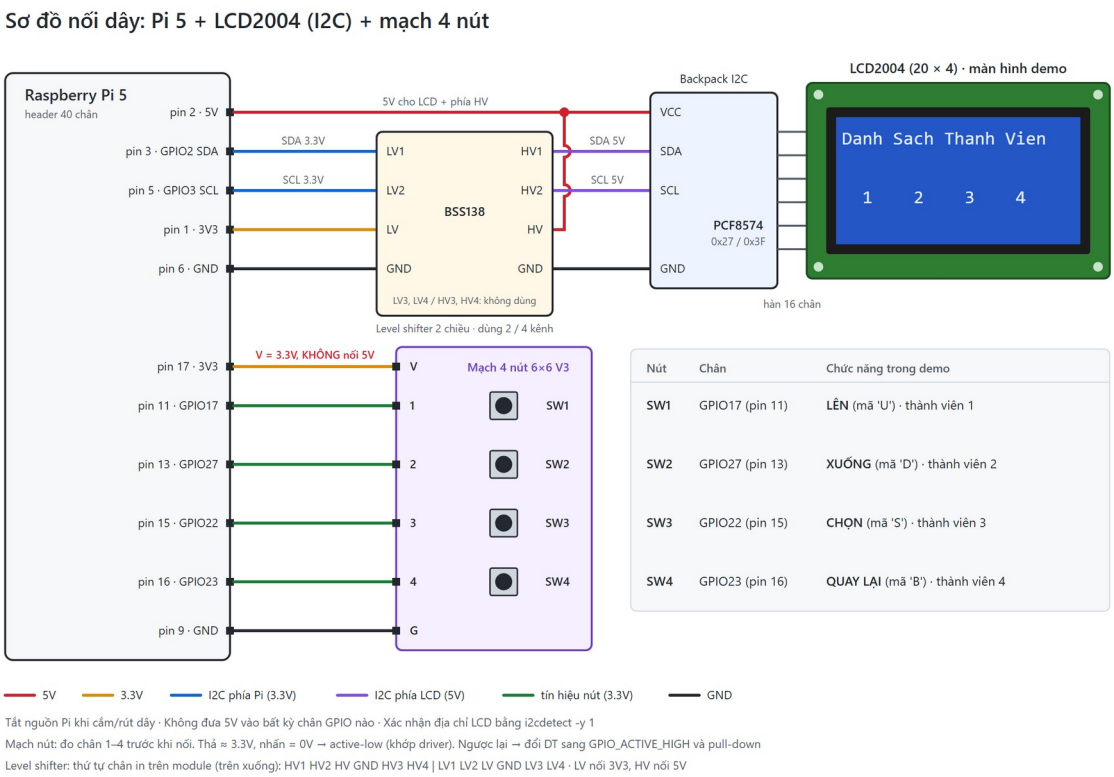


Figure 2. Wiring of the Raspberry Pi 5, level shifter, LCD board and buttons

1.3.5. Voltage protection

Raspberry Pi pins accept only 3.3 V. The LCD board runs at 5 V and pulls SDA and SCL up to 5 V. If we connect it directly, the bus voltage is about:

$$V = 3.3 + (5 - 3.3) \times 1.8 / (1.8 + 4.7) \approx 3.77\ V$$

(1.8 kΩ is the pull-up on the Pi, 4.7 kΩ is a typical pull-up on the LCD board.) This is too high. The BSS138 level shifter keeps the Pi side at 3.3 V and the LCD side at 5 V [16]. The buttons use only 3.3 V.

CHAPTER 2: SYSTEM DESIGN

2.1. System block diagram

Figure 3 shows the system in four parts: programs, device file, kernel and hardware. Programs only see `/dev/lcd2004`. The driver uses the kernel I2C functions for the LCD and the GPIO functions for the buttons. The overlay tells the kernel where the devices are.

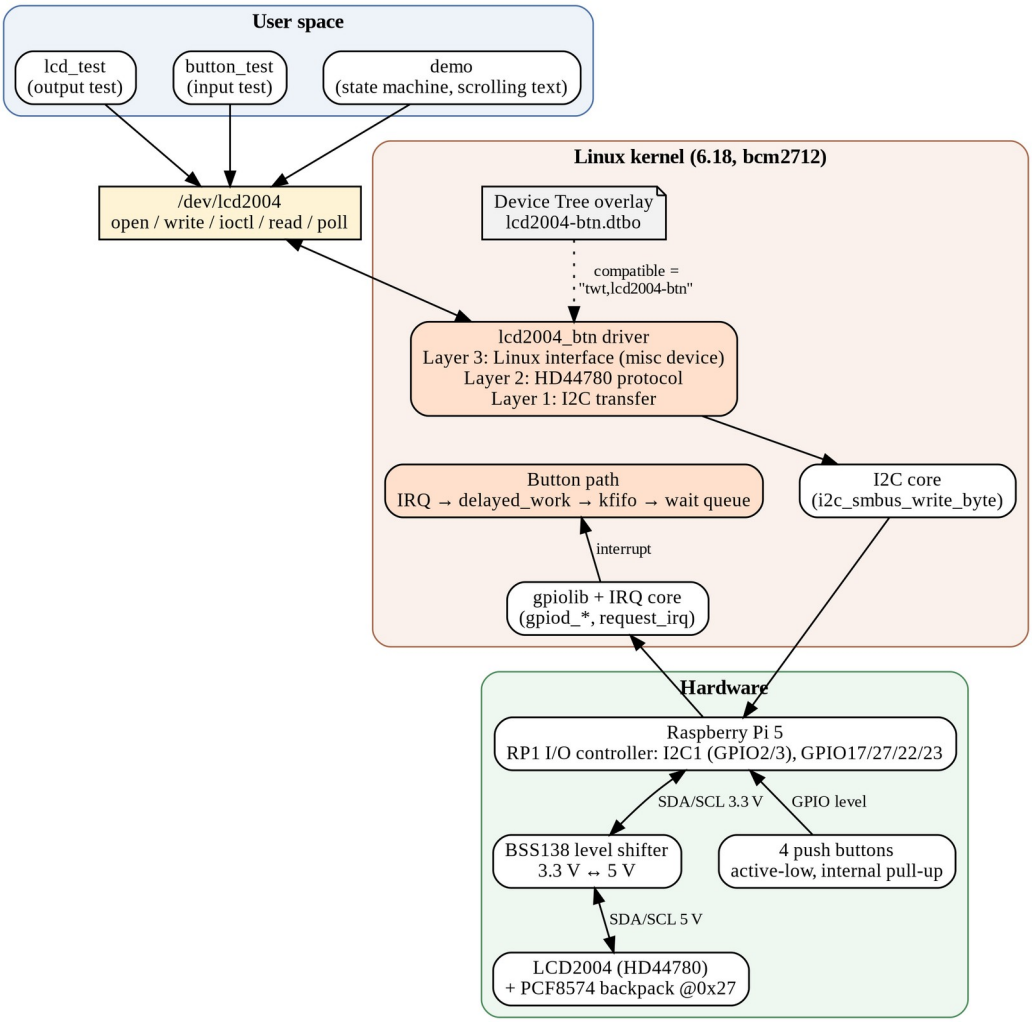


Figure 3. System block diagram

2.2. Driver structure

The driver is one C file, `driver/lcd2004_btn.c` (about 560 lines). It is split into parts (Table 7). Each part only calls the part below it, so each part is easy to explain.

Table 7. Parts of the driver

Part	Job	Main functions
1. I2C	Send one byte to the PCF8574	`pcf_write()`
2. HD44780	Start-up, send commands and	`lcd_init()`, `lcd_cmd()`,

Part	Job	Main functions
	characters, move the cursor	`lcd_putc()`, `lcd_goto()`
3. Linux interface	Functions of the device file	`lcd_write()`, `lcd_ioctl()`, `lcd_read()`, `lcd_poll()`
Buttons	Interrupt, debounce, event queue	`btn_isr()`, `btn_debounce()`
Start / stop	Set up and remove the device	`lcd_probe()`, `lcd_remove()`

All data of the device is kept in one structure, `struct lcd2004`. There are no global variables.

2.3. Device Tree overlay

The overlay `dts/lcd2004-btn-overlay.dts` does two things: it sets the four button pins as inputs with pull-up, and it adds the LCD to the I2C bus with address 0x27 and the list of button pins. The parameter `addr` allows another address, for example `dtoverlay=lcd2004-btn,addr=0x3f`.

Listing 1. LCD entry in the Device Tree overlay

```
lcd: lcd2004@27 {
    compatible = "twt,lcd2004-btn";   /* matches the driver */
    reg = <0x27>;                     /* I2C address */
    pinctrl-names = "default";
    pinctrl-0 = <&lcd_btn_pins>;      /* inputs with pull-up */
    /* button 1..4; flag 1 = active-low */
    button-gpios = <&gpio 17 1>, <&gpio 27 1>,
                   <&gpio 22 1>, <&gpio 23 1>;
    status = "okay";
};
```

2.4. Interface for programs

The file `include/lcd2004_ioctl.h` is shared by the driver and the programs, so both use the same command numbers and button codes. Table 8 lists what programs can do.

Table 8. Operations of /dev/lcd2004

Operation	What it does
`write()`	Prints text. `\n` goes to the next row. A full row continues on the next row.
`ioctl(LCD_IOC_CLEAR)`	Clears the screen
`ioctl(LCD_IOC_SET_CURSOR)`	Moves the cursor to (row 0–3, column 0–19)
`ioctl(LCD_IOC_BACKLIGHT)`	0 = backlight off, 1 = on
`read()`	Returns `'1'`–`'4'` for each button press. Waits if there is none.
`poll()`	Tells the program when a button event is ready

Listing 2. Main definitions in lcd2004_ioctl.h

```
struct lcd_pos { __u8 row; __u8 col; };

#define LCD_IOC_CLEAR      _IO('L', 0)
#define LCD_IOC_SET_CURSOR _IOW('L', 1, struct lcd_pos)
#define LCD_IOC_BACKLIGHT  _IOW('L', 2, int)

#define LCD_BTN_1 '1'   /* GPIO17 */
#define LCD_BTN_2 '2'   /* GPIO27 */
#define LCD_BTN_3 '3'   /* GPIO22 */
#define LCD_BTN_4 '4'   /* GPIO23 */
```

2.5. Button event path

Figure 4 shows what happens when a button is pressed:

1. Each edge on the pin calls the interrupt handler. It only restarts a 30 ms timer.
2. When the button stops bouncing, the timer ends and the debounce function reads the pin.
3. If the button is newly pressed, its code (`'1'`–`'4'`) is put into the queue.
4. A program waiting in `read()` or `poll()` wakes up and gets the code.

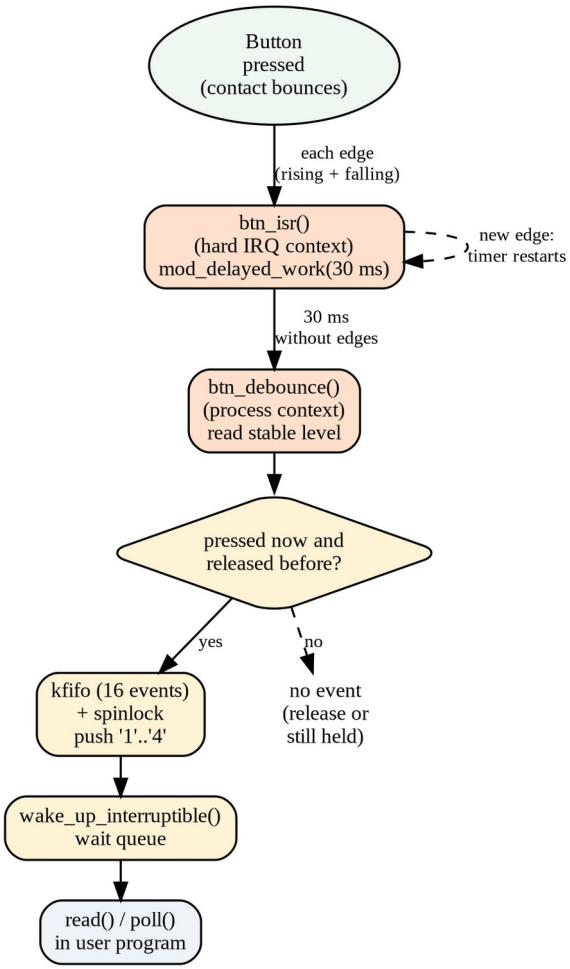


Figure 4. Path of a button event

2.6. Sharing and locking

Two programs can use the LCD at the same time, and button events can arrive while a program reads. Table 9 shows how the shared data is protected.

Table 9. Shared data and locks

Data	Used by	Protection
LCD and cursor position	`write()`, `ioctl()`	Mutex: one program writes at a time, texts do not mix
Event queue (`kfifo`)	Debounce function, `read()`	Spinlock (very short lock)
Pressed flag of a button	Only that button's debounce function	No lock needed

The LCD uses a mutex because I2C writes can sleep. The queue uses a spinlock because its operations are very short.

2.7. Start-up (probe) and removal

When the kernel finds the overlay entry, it calls `lcd_probe()` (Figure 5). The LCD is tested first, so a wiring error shows up early. The queue is prepared before the interrupts are turned on. The device file is created last, when everything is ready.

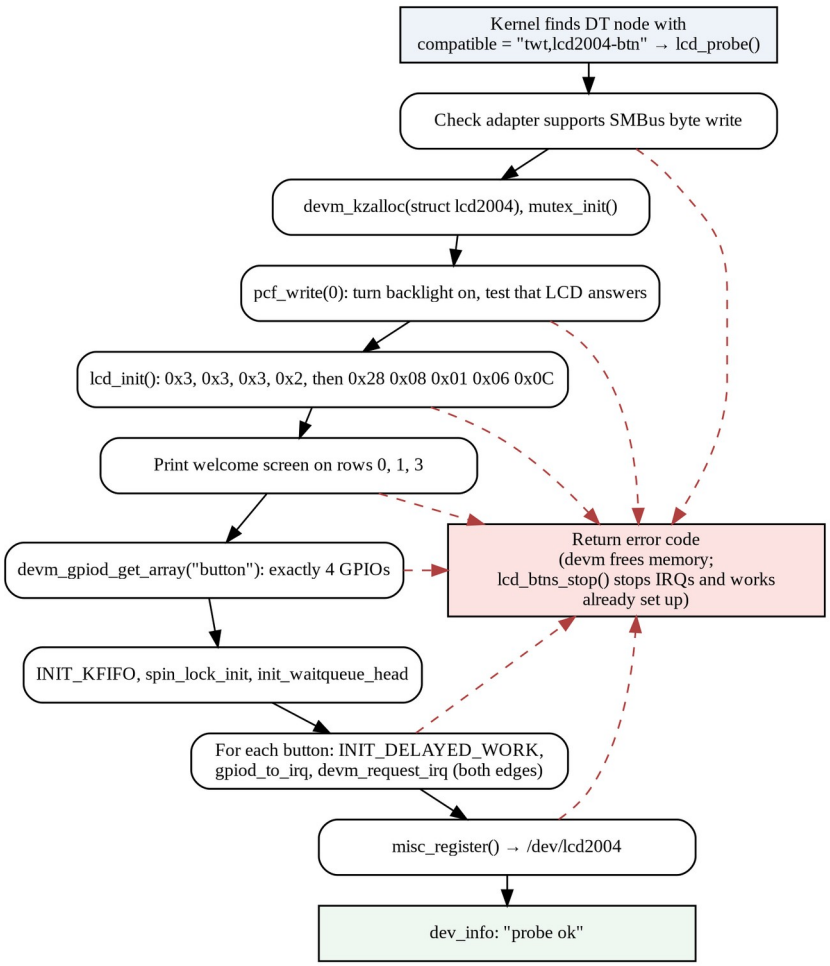


Figure 5. Steps of lcd_probe()

Memory, pins and interrupts are freed by the kernel automatically (`devm_*` functions). Before removal, `lcd_btns_stop()` turns off the interrupts and waits for pending work, so nothing runs on freed memory. `rmmod` is refused while a program still has `/dev/lcd2004` open.

2.8. Test and demo programs

Table 10. User programs

Program	What it does	Pass when
`lcd_test`	Prints on 4 rows, clears, moves the cursor, toggles the backlight, tries wrong values	It prints `ALL PASS`
`button_test`	Prints `NUT 1`…`NUT 4` with the time for each press	One line per press
`demo`	Buttons control what the LCD shows (Figure 6)	All steps work as designed

The demo has four states (Figure 6):

- **XIN_CHAO:** “Xin Chao” scrolls on the screen. Any button goes to the member list.
- **DSTV:** shows the list “1 2 3 4”. Button *k* selects member *k*.

- **SCROLL:** the name and student ID of member *k* scroll once.
- **INFO:** name and ID stay on the screen for 10 seconds, then the list comes back.

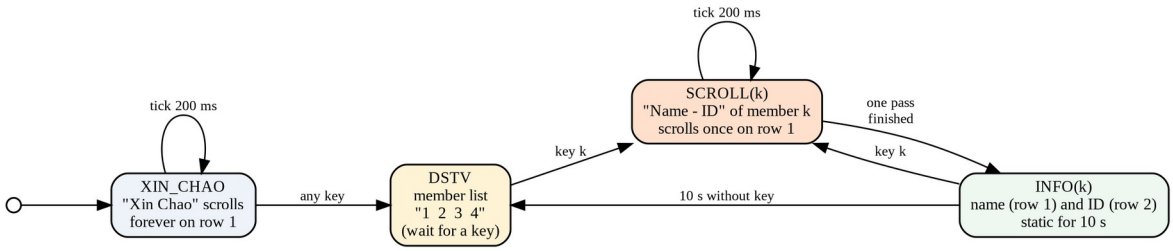


Figure 6. States of the demo program

The program uses one `poll()` loop, no threads. It waits for a button for at most 200 ms. If no button comes, it moves the text one step. Only the changed row is redrawn, so the screen does not flicker.

2.9. Built-in kernel design

For the "good" level, the driver is added to the kernel source in `drivers/auxdisplay/` (the folder for LCD drivers). The kernel is built on the Raspberry Pi 5 from the branch that matches the running kernel (`rpi-6.18.y`) [8]. Figure 7 shows the steps done by `tools/build-builtin.sh`.

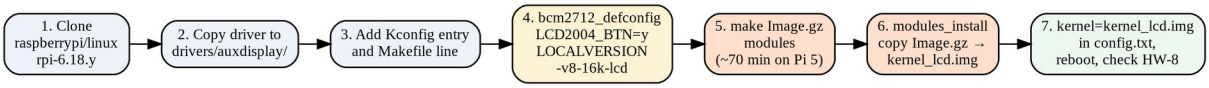


Figure 7. Steps to build the driver into the kernel

Safety: the new kernel has its own name, `kernel_lcd.img`, and is chosen by one line `kernel=kernel_lcd.img` in `config.txt`. Deleting this line brings back the old kernel. The kernel modules must still be installed, because the I2C controller of the Pi 5 is itself a module. The overlay is still needed: it tells the kernel where the device is.

Table 11. Module and built-in

	Module	**Built-in**
Where is the driver	`lcd2004_btn.ko` file	Inside `kernel_lcd.img`
How it starts	`sudo insmod` after each boot	By itself at boot
`lsmod`	Shows `lcd2004_btn`	Shows nothing
Remove	`rmmod`	Not possible
After a code change	Rebuild the module (seconds)	Rebuild the kernel and reboot

CHAPTER 3: IMPLEMENTATION AND SOURCE CODE

3.1. Development environment

Table 12. Development tools

Tool	Use
Raspberry Pi OS 64-bit, kernel 6.18	Build and run the driver, the programs and the kernel
`raspberrypi-kernel-headers`, `device-tree-compiler`, `i2c-tools`	Build the module, compile the overlay, find the I2C address
Windows 11 + WSL2 Ubuntu 24.04 + ARM64 compiler	Check that the code compiles before testing on the Pi
VS Code, SSH, `tmux`	Edit code, work on the Pi remotely, keep long builds running
Git and GitHub	Save versions and share code [17]

3.2. Hardware assembly

The circuit was built with the Pi unplugged: level shifter first, then power and ground, then I2C, then the buttons. Before power-on, the buttons and the power lines were checked with a multimeter, and every wire was compared with Table 6.

[INSERT PHOTO: the assembled circuit: Raspberry Pi 5, level shifter, LCD2004 and button board]

Figure 8. The assembled hardware

3.3. Building and loading the module

Listing 3. Commands for the module

```
# Overlay (once)
dtc -@ -I dts -O dtb -o lcd2004-btn.dtbo dts/lcd2004-btn-overlay.dts
sudo cp lcd2004-btn.dtbo /boot/firmware/overlays/
#   add to /boot/firmware/config.txt:  dtparam=i2c_arm=on
#                                      dtoverlay=lcd2004-btn
sudo reboot
i2cdetect -y 1                  # HW-1: address 27 must appear

# Driver and programs
make -C driver                  # -> lcd2004_btn.ko
sudo insmod driver/lcd2004_btn.ko
make -C app                     # -> lcd_test, button_test, demo
sudo rmmod lcd2004_btn          # HW-7: remove, then load again
```

3.4. Building the driver into the kernel

The script `tools/build-builtin.sh` runs on the Pi and does all steps of Figure 7: download the kernel source, copy the driver into `drivers/auxdisplay/`, add the Kconfig entry (Listing 4) and one Makefile line, turn the driver on (`=y`), build, install the modules and copy the new kernel. The option `--check` stops before the build to test the setup quickly. The script does not change `config.txt`; this is done by hand so everyone knows how to go back.

Listing 4. Kconfig entry added to drivers/auxdisplay/Kconfig

```
config LCD2004_BTN
     tristate "LCD2004 (PCF8574 I2C) + 4 buttons for Raspberry Pi 5"
     depends on I2C && GPIOLIB && OF
     help
       HD44780 20x4 character LCD behind a PCF8574 I2C backpack, plus
       4 push buttons. Creates /dev/lcd2004 (write, ioctl, read, poll).
```

`tristate` means the driver can be a module (`m`) or built-in (`y`). The full build took about 70 minutes on the 4 GB Pi 5. It was run inside `tmux`, so a lost SSH connection did not stop it.

3.5. Source code

The code below is taken from `driver/lcd2004_btn.c` and `app/demo.c`. Comments are translated into English.

3.5.1. Sending data to the LCD

Listing 5. Sending a byte to the LCD

```
static int pcf_write(struct lcd2004 *lcd, u8 data)
{
	return i2c_smbus_write_byte(lcd->client, data | lcd->backlight);
}

/* Send 4 bits with an E pulse so that the LCD reads them */
static int lcd_write4(struct lcd2004 *lcd, u8 nibble, u8 rs)
{
	u8 data = (nibble << 4) | rs;		/* 4 bits on P4..P7 */
	int ret;

	ret = pcf_write(lcd, data | LCD_EN);	/* E = 1 */
	if (ret)
		return ret;
	udelay(1);
	ret = pcf_write(lcd, data);		/* E = 0: LCD reads the data */
	if (ret)
		return ret;
	udelay(50);				/* wait for the LCD */
	return 0;
}

/* One byte = high 4 bits, then low 4 bits */
static int lcd_send(struct lcd2004 *lcd, u8 byte, u8 rs)
{
	int ret = lcd_write4(lcd, byte >> 4, rs);
	if (ret)
		return ret;
	return lcd_write4(lcd, byte & 0x0F, rs);
}
```

`pcf_write()` always adds the backlight bit, so the backlight never turns off by mistake. `lcd_write4()` sets the 4 bits and makes the E pulse. `lcd_send()` sends a byte in two halves. `lcd_goto()` moves the cursor using the row addresses `{0x00, 0x40, 0x14, 0x54}`.

3.5.2. write()

Listing 6. Main loop of lcd_write()

```
mutex_lock(&lcd->lock);                    /* one writer at a time */
while (done < count) {
     n = min_t(size_t, count - done, sizeof(buf));     /* 80 bytes at a time */
     if (copy_from_user(buf, ubuf + done, n)) { ret = -EFAULT; goto out; }
     for (i = 0; i < n; i++) {
            err = 0;
            if (buf[i] == '\n') {
                   err = lcd_newline(lcd);
            } else if (buf[i] >= 0x20 && buf[i] <= 0x7E) {
                   if (lcd->col >= LCD_COLS)      /* row full: next row */
                          err = lcd_newline(lcd);
                   if (!err)
                          err = lcd_putc(lcd, buf[i]);
            } else {
                   continue;                      /* skip other bytes */
            }
            if (err) { ret = -EIO; goto out; }
     }
     done += n;
}
out:
mutex_unlock(&lcd->lock);
```

Text is copied from the program 80 bytes at a time. A full row moves to the next row only when another character comes. Without this, `echo` of exactly 20 characters would skip one row (once for the full row, once for `\n`).

3.5.3. ioctl()

`lcd_ioctl()` first copies and checks the value from the program. A wrong row or column gives `EINVAL`, an unknown command gives `ENOTTY`. Only then does it lock the LCD and send the command. This keeps the lock short.

3.5.4. Interrupt and debouncing

Listing 7. Interrupt handler and debounce function

```
static irqreturn_t btn_isr(int irq, void *data)
{
	struct lcd_btn *btn = data;

	/* each edge restarts the 30 ms timer */
	mod_delayed_work(system_wq, &btn->work, msecs_to_jiffies(DEBOUNCE_MS));
	return IRQ_HANDLED;
}

static void btn_debounce(struct work_struct *w)
{
	struct lcd_btn *btn = container_of(to_delayed_work(w), struct lcd_btn, work);
	struct lcd2004 *lcd = btn->lcd;
	int v = gpiod_get_value_cansleep(btn->gpio);   /* 1 = pressed */

	if (v < 0)
		return;
	if (v && !btn->pressed) {                    /* new press */
		btn->pressed = true;
		if (kfifo_in_spinlocked(&lcd->events, &btn->code, 1, &lcd->fifo_lock))
			wake_up_interruptible(&lcd->wq);    /* wake the reader */
	} else if (!v) {
		btn->pressed = false;                    /* released: no event */
	}
}
```

The interrupt handler does almost nothing, which is how it should be. The debounce function makes one event per press. If the queue is full (16 events), new presses are dropped.

3.5.5. read() and poll()

Listing 8. lcd_read() and lcd_poll()

```
while (1) {
	n = kfifo_out_spinlocked(&lcd->events, tmp,
				 min_t(size_t, count, sizeof(tmp)), &lcd->fifo_lock);
	if (n)
		break;                                  /* got events */
	if (file->f_flags & O_NONBLOCK)
		return -EAGAIN;                         /* do not wait */
	if (wait_event_interruptible(lcd->wq, !kfifo_is_empty(&lcd->events)))
		return -ERESTARTSYS;                    /* Ctrl+C */
}
if (copy_to_user(ubuf, tmp, n))
	return -EFAULT;
return n;

/* poll(): ready when the queue is not empty */
poll_wait(file, &lcd->wq, pt);
return kfifo_is_empty(&lcd->events) ? 0 : EPOLLIN | EPOLLRDNORM;
```

`read()` sleeps until there is an event, so it uses no CPU while waiting. Ctrl+C stops the wait, so the test programs can exit cleanly.

3.5.6. probe() and remove()

`lcd_probe()` follows Figure 5. It checks that the LCD answers on I2C, starts the LCD and shows a welcome screen (“LCD2004 driver OK”). This screen proves the driver works even before any program runs. Then it gets the 4 button pins, turns on their interrupts and creates `/dev/lcd2004`. `lcd_remove()` stops the buttons, removes the device file and turns off the backlight.

3.5.7. Demo main loop

Listing 9. Main loop of the demo (shortened)

```
while (!stop && !err) {
     /* member list: wait for a button forever; other states: 200 ms */
     r = poll(&pfd, 1, d.state == DSTV ? -1 : TICK_MS);
     if (r < 0) {
            if (errno == EINTR)
                  continue;                    /* Ctrl+C */
            err = 1;
     } else if (r == 0) {
            err = on_tick(&d) != 0;           /* no button: move the text */
     } else {
            n = read(d.fd, buf, sizeof(buf)); /* one byte per press */
            for (i = 0; i < n && !err; i++)
                  err = on_key(&d, buf[i]) != 0;
     }
}
```

`on_key()` handles a button and `on_tick()` moves the scrolling text. When the user presses Ctrl+C, the program clears the LCD and exits.

CHAPTER 4: TESTING AND EVALUATION

4.1. Test method

A driver cannot be tested on the laptop, so we tested on the real board in eight steps, HW-1 to HW-8. Each step has a clear pass rule. We moved to the next step only after the current one passed, so any error points to one part.

4.2. Results with the module

Table 13. Test steps and results

Test	Pass rule	Result
HW-1	`i2cdetect -y 1` shows the LCD address	Passed (0x27)
HW-2	`insmod` works, `/dev/lcd2004` exists, welcome screen shows	Passed
HW-3	`echo "Hello" > /dev/lcd2004` shows the text	Passed
HW-4	`lcd_test` prints `ALL PASS`	Passed
HW-5	`button_test`: all 4 buttons work, one event per press	Passed
HW-6	`demo` works as in Figure 6	Passed
HW-7	`rmmod` then `insmod` again without error	Passed
HW-8	Built-in kernel: `/dev/lcd2004` exists without `insmod`	Passed (4.3)

All eight steps passed on the Raspberry Pi 5 (HW-8 on 21 October 2026). `button_test` printed one line per press, so the 30 ms debounce works.

[INSERT PHOTO: LCD showing the welcome screen after insmod (or after boot)]

Figure 9. Welcome screen from lcd_probe()

[INSERT PHOTO: LCD during lcd_test, and the terminal showing ALL PASS]

Figure 10. lcd_test result

[INSERT PHOTO: demo screens: member list "1 2 3 4" and one member's details]

Figure 11. Demo controlled by the buttons

4.3. Results with the built-in kernel

After switching to `kernel_lcd.img` and rebooting, without `insmod`:

Table 14. Checks of the built-in driver (HW-8)

Check	Result	Meaning
Look at the LCD	Welcome screen shows by itself	Driver starts at boot
`uname -r`	`6.18.57-v8-16k-lcd`	Our kernel is running
`lsmod | grep lcd2004`	Nothing	Driver is not a module
`grep lcd2004 .../modules.builtin`	`kernel/drivers/auxdisplay/lcd2004_btn.ko`	Driver is inside the kernel
`ls /dev/lcd2004`	File exists	Device file was created
`lcd_test`, `button_test`, `demo`	Same as with the module	Programs need no change

[INSERT PHOTO: terminal output of uname -r, lsmod, modules.builtin and ls /dev/lcd2004 on the built-in kernel]

Figure 12. Terminal output of the HW-8 checks

4.4. Comparison with the course lab guide

Table 15. Our choices compared with the lab guide

Topic	Lab guide [2]	This project	Why
Button pins	Fixed number `gpio_request(17)`	Pins from Device Tree	Numbers differ on the Pi 5
I2C address	Fixed in the code	In the overlay	Change address without rebuilding
Device file	4 steps (`cdev`, `class_create`)	`misc_register()`	Simpler
Interrupt	All work in the handler	Handler only starts a timer	Handler stays short
Button events	`read()` returns a text	Queue, `read()`, `poll()`	No lost presses, no busy waiting

4.5. Overall assessment

Both levels of the assignment were reached. The driver works as a module and inside the kernel, with the same code. The test programs check the LCD and the buttons separately, and the demo shows the buttons controlling the LCD. The circuit protects the Pi from 5 V, and the old kernel can always be restored.

4.6. Known limitations

- If the queue already has 16 events, new presses are lost.
- After an I2C error (for example a loose wire), the cursor position may be wrong; clear the screen to fix it.
- Only ASCII characters are shown; Vietnamese accents are skipped.
- Removing the device through sysfs while it is open is not safe (normal `rmmod` is safe).

- The built-in driver cannot be removed; any code change needs a kernel rebuild.

4.7. Difficulties and solutions

Table 16. Difficulties and solutions

Difficulty	Solution
LCD board uses 5 V, Pi pins use 3.3 V	BSS138 level shifter
Old GPIO numbers do not work on the Pi 5	Take the pins from the Device Tree
One press gave several events	30 ms debounce timer
`echo` of 20 characters skipped one row	Go to the next row only when another character comes
The Pi runs kernel 6.18, not 6.12	Script picks the kernel branch from `uname -r`
New kernel without modules has no I2C	Install the modules with a new version name
Risk that the Pi does not boot	Separate `kernel_lcd.img`; delete one line to go back
Kernel build takes about 70 minutes	Run it inside `tmux`

CONCLUSION

We built a Linux driver for an LCD2004 and four push buttons on a Raspberry Pi 5. The driver controls the LCD over I2C, reads the buttons with interrupts and debouncing, and gives programs one simple file, `/dev/lcd2004`. The pins and the address are in a Device Tree overlay. The driver works both as a module and built into the kernel, so both levels of the assignment are met. All eight hardware tests passed.

We learned how a driver is linked to hardware through the Device Tree, why interrupt handlers must be short, how to protect shared data, and how to build and change the kernel of a board safely.

Future work:

- Show Vietnamese letters with custom characters.
- Detect long presses.
- Fix the known limitations.
- Use the standard kernel LCD and input frameworks.

REFERENCES

[1] Trương Ngọc Sơn, *Embedded System*, lecture notes, HCMC University of Technology and Education, 2026.

[2] Course laboratory guide, *Linux driver on Raspberry Pi 5* (exercises 2.1–2.5), Embedded Systems course, HCMUTE, 2026.

[3] Hitachi, *HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver*, datasheet.

[4] NXP Semiconductors, *PCF8574; PCF8574A Remote 8-bit I/O expander for I2C-bus with interrupt*, data sheet.

[5] NXP Semiconductors, *UM10204 I2C-bus specification and user manual*.

[6] Raspberry Pi Ltd, *Raspberry Pi hardware documentation*. https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

[7] Raspberry Pi Ltd, *RP1 Peripherals*, datasheet, 2023.

[8] Raspberry Pi Ltd, *The Linux kernel*. https://www.raspberrypi.com/documentation/computers/linux_kernel.html

[9] Raspberry Pi Ltd, *Device Trees, overlays and parameters*. https://www.raspberrypi.com/documentation/computers/configuration.html

[10] The Linux Kernel documentation, *GPIO Descriptor Consumer Interface*.

[11] The Linux Kernel documentation, *Implementing I2C device drivers*.

[12] The Linux Kernel documentation, *Building External Modules* and *Kconfig Language*.

[13] J. Corbet, A. Rubini and G. Kroah-Hartman, *Linux Device Drivers*, 3rd ed., O'Reilly Media, 2005.

[14] Linux kernel source, drivers/auxdisplay/hd44780.c.

[15] Linux kernel source, Documentation/devicetree/bindings/input/gpio-keys.yaml.

[16] NXP Semiconductors, *AN10441 Level shifting techniques in I2C-bus design*, application note.

[17] Project source code: https://github.com/AnNguen-TwT-Projects/Pi5-Driver-Project